

The Python **os** Module

A Minimal, Complete Reference Guide

Covers paths, directories, files, environment variables, processes, and permissions — with short, runnable examples.

Table of Contents

1. Introduction
2. Importing the Module
3. Working Directory
4. Listing and Walking Directories
5. Creating and Removing Directories
6. File Operations
7. Path Handling (os.path)
8. Environment Variables
9. Running Processes
10. File Permissions and Stats
11. Useful Constants
12. Quick Reference Table

1. Introduction

The **os** module provides a portable way of using operating system dependent functionality, such as reading or writing to the file system, managing directories, handling environment variables, and running processes. It works consistently across Windows, macOS, and Linux.

*Related module: **os.path** is used for path manipulation and is covered separately in Section 7.*

2. Importing the Module

```
import os
```

3. Working Directory

Get and change the current working directory.

```
# Get current working directory
cwd = os.getcwd()
print(cwd)

# Change working directory
os.chdir("/path/to/folder")
```

4. Listing and Walking Directories

List entries in a directory:

```
entries = os.listdir(".")
print(entries) # ['file1.txt', 'folder', 'file2.py']
```

Recursively walk a directory tree with **os.walk()**, which yields the root path, subdirectories, and files at each level:

```
for root, dirs, files in os.walk("."):
    print("Directory:", root)
    for f in files:
        print("  File:", f)
```

os.scandir() is a faster alternative to os.listdir() that returns DirEntry objects with cached file-type information.

```
with os.scandir(".") as entries:
    for entry in entries:
        print(entry.name, entry.is_file())
```

5. Creating and Removing Directories

```
# Create a single directory
os.mkdir("new_folder")

# Create nested directories
```

```
os.makedirs("parent/child/grandchild", exist_ok=True)
```

```
# Remove an empty directory  
os.rmdir("new_folder")
```

```
# Remove nested empty directories  
os.removedirs("parent/child/grandchild")
```

Note: `os.rmdir()` and `os.removedirs()` only work on empty directories. Use `shutil.rmtree()` to delete a non-empty directory tree.

6. File Operations

```
# Rename or move a file  
os.rename("old_name.txt", "new_name.txt")
```

```
# Remove a file  
os.remove("file_to_delete.txt")
```

```
# Check if a path exists  
os.path.exists("file.txt")
```

7. Path Handling (os.path)

os.path handles path strings in a platform-independent way — always prefer it over manually joining strings with slashes.

```
# Join path components
path = os.path.join("folder", "subfolder", "file.txt")

# Split into directory and file name
dir_name, file_name = os.path.split(path)

# Get file extension
name, ext = os.path.splitext("script.py") # ('script', '.py')

# Absolute path
abs_path = os.path.abspath("file.txt")

# Checks
os.path.isfile(path)
os.path.isdir(path)
os.path.exists(path)
```

8. Environment Variables

```
# Read an environment variable safely
value = os.environ.get("PATH")

# Set an environment variable (current process only)
os.environ["MY_VAR"] = "123"

# Read directly (raises KeyError if missing)
home = os.environ["HOME"]
```

9. Running Processes

```
# Run a shell command
os.system("echo Hello")

# Get the current process ID
pid = os.getpid()
```

For capturing command output or advanced process control, use the subprocess module instead of os.system().

10. File Permissions and Stats

```
# Get file metadata
info = os.stat("file.txt")
print(info.st_size) # size in bytes
print(info.st_mtime) # last modified time
```

```
# Change file permissions (Unix-style octal mode)
os.chmod("file.txt", 0o755)
```

11. Useful Constants

```
os.sep      # path separator, e.g. '/' or '\\'
os.linesep  # line ending, e.g. '\n' or '\r\n'
os.name     # 'posix', 'nt', etc.
```

12. Quick Reference Table

Function	Purpose
<code>os.getcwd()</code>	Get current working directory
<code>os.chdir(path)</code>	Change working directory
<code>os.listdir(path)</code>	List directory contents
<code>os.walk(path)</code>	Recursively traverse a directory tree
<code>os.mkdir(path)</code>	Create a single directory
<code>os.makedirs(path)</code>	Create nested directories
<code>os.rmdir(path)</code>	Remove an empty directory
<code>os.remove(path)</code>	Delete a file
<code>os.rename(src, dst)</code>	Rename or move a file
<code>os.path.join(...)</code>	Join path components portably
<code>os.path.exists(path)</code>	Check if a path exists
<code>os.environ</code>	Access environment variables
<code>os.system(cmd)</code>	Run a shell command
<code>os.stat(path)</code>	Get file metadata
<code>os.chmod(path, mode)</code>	Change file permissions

End of guide.